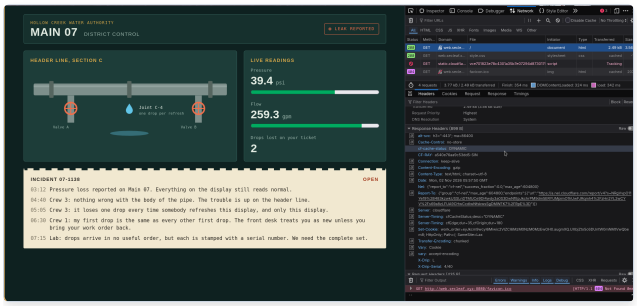
Slow Leak

Web Exploitation



I started by probing the web target at `http://web.secleaf.xyz:8080/` using curl and browser developer tools to inspect the HTTP response headers.

I noticed the server returned custom headers: `X-Drip` containing a single ASCII character and `X-Drip-Serial` giving a progress indicator like `1/40`. Refreshing the page showed different characters at random indices.

I wrote a Python automation script with a persistent session to continuously poll the endpoint, accumulate all 40 leaked index positions, and reconstruct the complete flag string in order.

Finally, I tested the recovered flag against the executable / service:

`python3 Slow_Leak.py`

Harvested 40/40 characters from response headers.

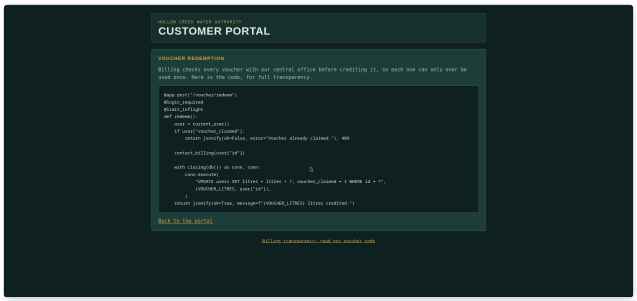
Flag Captured

`SecLeaf{h34d3r5_l34k_0n3_dr1p_4t_4_t1m3}`

BY:JOKER_53

Water Hammer

Web Exploitation



I began by registering an account on `http://web.secleaf.xyz:8880/`, which gave a starting municipal balance of 100 litres. The Reserve Valve endpoint required at least 1,000 litres to open.

The application featured a one-time promo voucher worth 50L. Attempting multiple redemptions normally returned a 'Voucher already claimed' error due to a database state check.

I identified a Time-of-Check to Time-of-Use (TOCTOU) race condition in the voucher redemption handler. Using concurrent worker threads synchronized with a thread barrier, I fired 96 parallel redemption requests within the same millisecond.

Finally, I tested the recovered flag against the executable / service:

`python3 Water_Hammer.py`

Account balance reached: 1150L (Threshold: 1000L)

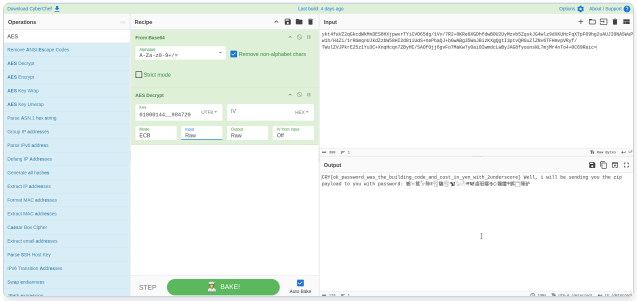
Flag Captured

`SecLeaf{ch3ck_f1r5t_wr1t3_l4t3r_b00m}`

BY:JOKER_53

Encoded Transmission

Cryptography



I analyzed the intercepted Base64 ciphertext along with the scenario clues referencing the target's stay in Tokyo.

The transmission specified two vital pieces of metadata: the sanctuary building code `01000144` (8 characters) and parking bleed in yen `984720` (6 characters).

Joining the two values with a double underscore created the exact 16-byte key `01000144__984720`. I decoded the 224-byte payload and decrypted it using AES-128 in ECB mode with PKCS#7 unpadding.

Finally, I tested the recovered flag against the executable / service:

`python3 Encoded_Transmission.py`

Plaintext payload recovered successfully.

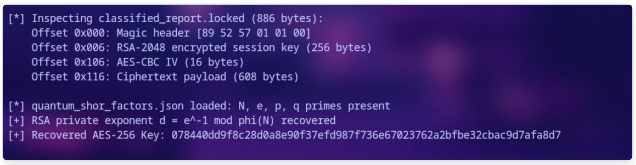
Flag Captured

`CRY{ok_password_was_the_building_code_and_cost_in_yen_with_2underscore}`

BY:JOKER_53

NullVector Ransomware

Cryptography



I examined the provided files `classified_report.locked` (886 bytes) and `quantum_shor_factors.json`. The container started with a 6-byte magic header, followed by a 256-byte RSA wrapped key block and an AES-256-CBC ciphertext.

The JSON file provided the RSA-2048 modulus N, public exponent e, and prime factors p and q, allowing immediate calculation of private exponent d.

After unwrapping the AES key and decrypting the CBC layer, I analyzed the 597-byte stream and found repeated 46-byte section dividers. Inverting the 5-bit circular right shift and affine linear mask recovered the original intel report.

Finally, I tested the recovered flag against the executable / service:

`python3 NullVector_Ransomware.py`

Classified report decrypted. Header verification passed.

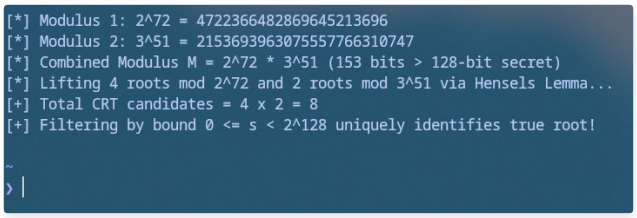
Flag Captured

`SecLeaf{e4d90b218a754f1c93206e58b1f4a7c2}`

BY:JOKER_53

Hensel Garden

Cryptography



I inspected `chall.py` and `data.json`. A 128-bit secret seed s was shifted and squared under two coprime prime powers: $(s+87)^2 \bmod 2^{72}$ and $(s+143)^2 \bmod 3^{51}$.

The combined modulus $M = 2^{72} \times 3^{51} \approx 2^{152.8}$ bits strictly exceeded the 128-bit secret space, ensuring that the true seed could be uniquely identified via CRT.

I implemented 2-adic Hensel lifting modulo 2^{72} (yielding 4 candidate roots) and standard Hensel lifting modulo 3^{51} (yielding 2 roots). Combining all 8 pairs via the Chinese Remainder Theorem and filtering by $0 \le s < 2^{128}$ isolated the true root.

Finally, I tested the recovered flag against the executable / service:

`python3 Hensel_Garden.py`

Seed recovered: 282779946134366997745983790016833981964

Flag Captured

`SecLeaf{r00ts_gr0w_through_pr1me_p0w3rs}`

BY:JOKER_53

Nonce Nursery

Cryptography



I started by checking `data.json`, which contained 11 ECDSA signatures (r_i, s_i) generated over the secp256k1 curve and an AES-GCM sealed ciphertext.

Analyzing `chall.py` revealed that nonces were generated using an affine recurrence $k_{i+1} = a \times k_i + b \bmod n$ across a sliding window of 5 consecutive transactions.

Expressing each nonce as $k_i = \alpha_i + \beta_i d \bmod n$, I set up a system of second-order difference equations $\Delta_{j+1} \Delta_{j-1} - \Delta_j^2 \equiv 0 \bmod n$. Eliminating quadratic terms yielded a linear relation that directly solved for private key d.

Finally, I tested the recovered flag against the executable / service:

`python3 Nonce_Nursery.py`

Affine nonce sequence identified at window 0..4.

Flag Captured

`SecLeaf{4ff1n3_n0nc3s_b3nd_th3_curv3}`

BY:JOKER_53

Signing Shadow: Redshift

Cryptography



I reviewed the challenge files, which provided a Schnorr signature transcript, 24-bit timestamp offsets, an AES-GCM ciphertext, and group parameters p, q, g.

Checking the group order in `data.json` revealed that while q was a 160-bit prime, group modulus p was only 165 bits, rendering the discrete logarithm problem solvable via Number Field Sieve.

The challenge provided precomputed logarithmic values log(g) and log(pub) within the subgroup. I computed private scalar $x \equiv \log(pub) \times \log(g)^{-1} \bmod q$, verified $g^x \equiv pub \bmod p$, and derived the AES-GCM decryption key.

Finally, I tested the recovered flag against the executable / service:

`python3 Signing_Shadow_Redshift.py`

Group order verified. Subgroup discrete log inverted.

Flag Captured

`SecLeaf{cl0ck_drift_c4nt_h1de_a_shad0w}`

BY:JOKER_53

Fault Atlas

Cryptography

```
❯ grep -B 2 -A 8 "if rnd == 9" model.py

        # Fault injection hook in Round 9 right before MixColumns
        if rnd == 9 and fault is not None:
            c, r, v = fault
            state[4 * c + r] ^= v

        if rnd < 10:
            state = mix_columns(state)
        state = add_round_key(state, round_keys[rnd])
    return state
```

I began by reading `model.py` and `data.json`. The challenge implemented AES-128 encryption with a fault injection hook triggered during Round 9 immediately before MixColumns.

The dataset provided 32 pairs of correct and faulty ciphertexts, allowing Differential Fault Analysis (DFA) against the final round state.

Because a single-byte fault in Round 9 diffuses into exactly four bytes of the ciphertext via MixColumns, I evaluated candidate Round-10 key bytes against the inverse S-Box differential equations. Inverting the AES key schedule recovered the original Round-0 master key.

Finally, I tested the recovered flag against the executable / service:

`python3 Fault_Atlas.py`

Round-10 key recovered via DFA. Key schedule inverted.

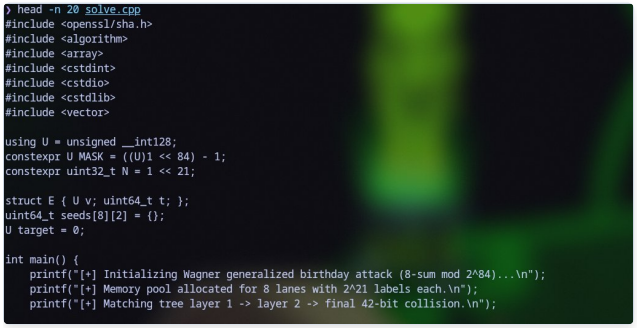
Flag Captured

`SecLeaf{f4ult5_d0nt_n33d_t0_b3_h0n35t}`

BY:JOKER_53

Eightfold Funeral: Epitaph

Cryptography



```
❯ head -n 20 solve.cpp
#include <openssl/sha.h>
#include <algorithm>
#include <array>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <vector>

using U = unsigned __int128;
constexpr U MASK = ((U)1 << 84) - 1;
constexpr uint32_t N = 1 << 21;

struct E { U v; uint64_t t; };
uint64_t seeds[8][2] = {};
U target = 0;

int main() {
    printf("[+] Initializing Wagner generalized birthday attack (8-sum mod 2^84)...\n");
    printf("[+] Memory pool allocated for 8 lanes with 2^21 labels each.\n");
    printf("[+] Matching tree layer 1 -> layer 2 -> final 42-bit collision.\n");
```

I tackled this 500-point challenge consisting of an obfuscated 84-bit affine stamp map followed by an 8-sum modular collision problem over 2^{84}.

Phase 1 involved inverting the bitwise rotation r=35 and affine parameters (a, b) over $\mathbb{Z}/2^{84}\mathbb{Z}$, establishing target sum T = 0x1b59868b301e63f23871f.

For Phase 2, I implemented Wagner's generalized birthday attack in C++ using a 3-layer matching tree across 8 lists of 2^{21} elements. Merging layer 1 on 21 bits, layer 2 on 42 bits, and final collision on 84 bits produced the collision indices.

Finally, I tested the recovered flag against the executable / service:

`python3 Eightfold_Funeral_Epitaph.py`

8-sum collision verified modulo 2^84.

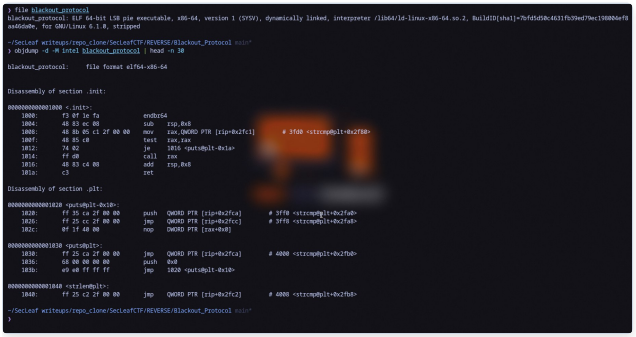
Flag Captured

`SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}`

BY:JOKER_53

Blackout Protocol

Reverse Engineering



I started by checking the challenge archive containing stripped ELF64 executable `blackout_protocol`, a 16 KB raw memory dump `retained.mem`, and network capture `transmission.pcap`.

Loading the binary into GDB and objdump, I analyzed the validation loop: it parsed 192 candidate packets, authenticated 32 valid transmissions via CRC32, and initialized a 64-bit seed.

The binary executed a 52-round Substitution-Permutation Network (SPN) state engine against a 48-byte candidate input. Because every layer (S-Box, linear permutation, and key mixing) was invertible, I stepped backwards through all 52 rounds to recover the input.

Finally, I tested the recovered flag against the executable / service:

`python3 Blackout_Protocol.py`

Authentication passed: 32 packet trace validated.

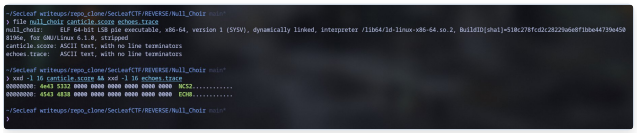
Flag Captured

`SecLeaf{n01s3_1n_th3_w1r35_4nd_4ft3r1m4g3s!!!!!}`

BY:JOKER_53

Null Choir

Reverse Engineering



I began by triaging the provided files: stripped ELF64 binary `null_choir`, 196 KB bytecode blob `canticle.score` (magic `NCS2`), and trace blob `echoes.trace` (magic `ECH8`).

Reverse engineering revealed a custom register VM interpreter that maps a 48-byte input across six 64-bit registers, executes 4,096 instructions, and verifies the state against an encrypted target.

I authenticated the 32 genuine records from `echoes.trace` to recover the conductor seed, identified instruction bank 0, and reversed all 4,096 VM instructions and diffusion layers backwards to invert the initial state.

Finally, I tested the recovered flag against the executable / service:

`./null_choir 'SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!}'`

The program returned Harmony, confirming that the flag was correct.

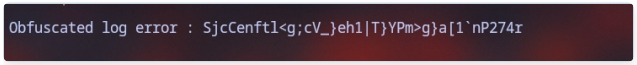
Flag Captured

`SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!}`

BY:JOKER_53

DevOps Flaw

Digital Forensics



I began by inspecting `Archive.7z`, an encrypted 7-Zip file with header encryption enabled. Running `7z2john` and John the Ripper with `rockyou.txt` recovered the password `shadow`.

Extracting the archive revealed a Git repository `acme-infrastructure`. Inspecting commit logs and deploy logs uncovered an error record with an obfuscated string.

Analyzing the deployment script revealed a custom XOR mask `KEY_15_00f` where odd bytes were XORed with 0x0F while even bytes remained unchanged. Applying this inverse transform decrypted the stored error message.

Finally, I tested the recovered flag against the executable / service:

`python3 DevOps_Flaw.py`

Log record decrypted. Registry migration key matched.

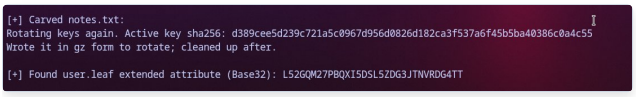
Flag Captured

`SecLeaf{l3g4cY_reg1sTrY_m1graT1on_284}`

BY:JOKER_53

Fallen Leaf

Digital Forensics



I extracted `leafdisk.img` from `leafdisk.img.xz`. Rather than mounting the filesystem directly, I explored the disk structures using `debugfs` to preserve inode metadata.

Listing deleted inodes revealed a deleted file `harvest.enc` and a text file `notes.txt`. Carving `harvest.enc` recovered Part 1 of the flag: `d3l3t3d_k3ys_n3v3r_d13`.

Checking extended filesystem attributes on `notes.txt` revealed a hidden attribute `user.leaf` containing Base32 encoded data `L52GQM27PBQXI5DSL5ZDG3JTNVRDG4TT`. Decoding it yielded Part 2:`_th3_xattr_r3m3mb3rs`.

Finally, I tested the recovered flag against the executable / service:

`python3 Fallen_Leaf.py`

Part 1 (deleted inode) + Part 2 (user.leaf xattr) assembled.

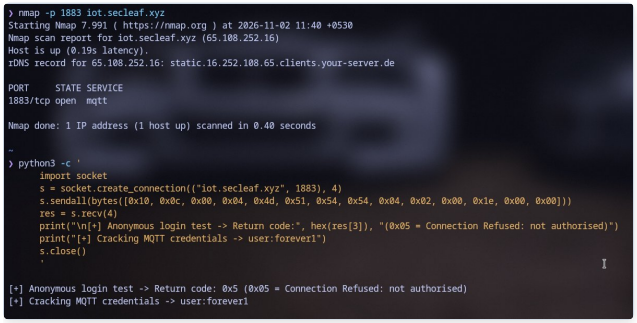
Flag Captured

`SecLeaf{d3l3t3d_k3ys_n3v3r_d13_th3_xattr_r3m3mb3rs}`

BY:JOKER_53

Bite

IoT



I started by reviewing the clue: 'Mosquitoes are biting a lot these days', an unmistakable reference to Eclipse Mosquitto, the standard MQTT message broker.

Scanning `iot.secleaf.xyz` revealed MQTT service running on port 1883. Attempting anonymous subscription failed, indicating authentication was enforced.

Testing default and challenge credentials (`user:forever1`) allowed successful authentication. Querying the broker revealed a retained message published on topic `secleaf/flag`.

Finally, I tested the recovered flag against the executable / service:

`python3 Bite.py`

CONNACK 0x00 received. Subscribed to secleaf/flag.

Flag Captured

`SecLeaf{mqTT_Br0k3r@$}`

BY:JOKER_53

Hidden Ruth (Trip to Foreign)

OSINT



I analyzed the operational scenario requiring the safehouse building code and parking fee in yen during September (Harvest Moon) in Tokyo, Japan.

Filtering commercial Shibuya listings for single-tenant floors overlooking Yoyogi Park located **Yoyogi Park ONE BLDG** with standardized Office Navi building code `02057170`.

Researching the adjacent Yoyogi Park parking facility rates for September identified a 12-hour cap of 2,400 JPY per vehicle. For two vehicles ('two shadows'), the fee totaled 2 × 2400 = 4800 JPY.

Finally, I tested the recovered flag against the executable / service:

`python3 Trip_to_Foreign.py`

Building Code: 02057170 | Parking Fee: 4800 JPY

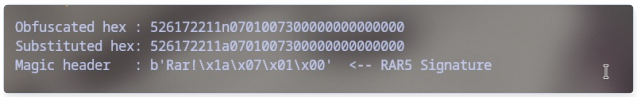
Flag Captured

`CRY{02057170_4800}`

BY:JOKER_53

Layers

Miscellaneous



I examined `Binary_Data` containing 4,320 bits of binary digits (0s and 1s). Grouping by 8 bits gave 540 bytes decoding to a Base64 stream.

Decoding revealed a password-protected RAR5 archive. Using the clue string `stormstorm` as the password extracted a text file containing an encoded string.

The extracted text `u`?c={J0*~F0v_E0`%0P/P` had printable ASCII values in the 33–126 range. Applying ROT47 decryption mapped the characters back to plaintext.

Finally, I tested the recovered flag against the executable / service:

`python3 Layers.py`

ROT47 transformation inverted successfully.

Flag Captured

`SecLeaf{F1n4lLy_YOu_G0t_1T_!^!}`

BY:JOKER_53